

REACT PRACTICE BLUEPRINT

WorkNest

Multi-role Project & Task Management Dashboard

Goal Ek hi project ko 10 structured phases me build karke React fundamentals, real-world architecture, advanced patterns, testing aur future API integration clear karna.

Aapka background	Is roadmap ka focus
Laravel • 4.5 years	Frontend thinking, component design, state flow
Daily study: 2 hours	12–14 weeks ka practical schedule
DOM: basic only	React-driven UI; direct DOM manipulation nahi
Backend later	Pehle mock data; phir Laravel/Spring Boot APIs

Is document ko kaise use karen

- Har phase ko sequence me complete karen; phase skip na karen.
- Codex se code mangne se pehle khud 20–30 minute solution sochiye.
- Har prompt ke baad generated code ko line-by-line samjhiye aur khud modify kijiye.
- “Done” tab maanẽ jab acceptance checklist manually pass ho.

1. Project Idea & Real-world Flow

WorkNest ek SaaS-style workspace app hoga jahan organizations apne projects, team members, tasks, deadlines aur activity ko manage karenge. UI pehle local mock data par chalega. Backend/API baad me same contracts ke through integrate hoga.

Primary roles

Role	Permissions
Workspace Admin	Workspace settings, members, all projects, reports
Project Manager	Project create/edit, members assign, tasks manage
Team Member	Assigned tasks view/update, comments add
Viewer	Read-only dashboards and project details

Main modules

- Authentication UI: login, forgot password, protected routes
- Dashboard: summary cards, recent activity, workload chart
- Projects: list/grid, filters, search, create/edit, project detail
- Kanban Board: To Do → In Progress → Review → Done
- Task Detail: subtasks, priority, assignee, due date, comments
- Team & Roles: member table, invitations, role-based UI
- Notifications & Settings: preferences, theme, profile
- Reports: project progress, task status, workload charts

End-to-end user flow

Step	User action	React learning
1	Login form submit	Controlled inputs, validation
2	Protected dashboard open	Routing, auth context
3	Project search/filter	Derived state, debouncing
4	Create project	Reusable form, reducer
5	Open project board	Nested routes, selectors
6	Move task	Immutable updates, drag/drop

7	Edit task/comments	Modal, async simulation
8	View report	Memoization, charts
9	Change role/theme	Context, persistence

2. React Concepts Coverage Map

Level	Concepts	WorkNest feature
Basic	JSX, components, props, events, conditional rendering	Sidebar, cards, status badges
Core	useState, lists/keys, forms, lifting state	Filters, project/task forms
Intermediate	Router, Context, useReducer, custom hooks	Auth, theme, app state
Advanced	Redux Toolkit, async state, memoization, lazy loading	Board, reports, route chunks
Professional	Testing, accessibility, error boundaries, architecture	Reliable production-style app
Integration	API service, auth token, loaders, errors	Laravel/Spring Boot connection

Important Redux ko start me mat lagaiye. Pehle props → lifting state → Context/useReducer ka pain samjhiye; uske baad Redux Toolkit ka reason naturally clear hoga.

3. Recommended Tech Stack

Category	Start with	Advanced phase
Build	Vite + React + JavaScript	Optional TypeScript migration
Styling	CSS Modules / plain CSS	Tailwind optional
Routing	React Router	Lazy routes
Forms	Controlled inputs	React Hook Form + Zod

State	useState + Context/useReducer	Redux Toolkit
Server state	Mock Promise service	TanStack Query
Charts	Simple CSS first	Recharts
Testing	Vitest + Testing Library	MSW + E2E optional
Quality	ESLint + Prettier	Husky optional

Suggested folder structure

- src/app — providers, router, store
- src/assets — icons/images
- src/components — shared UI components
- src/features/auth, projects, tasks, team, reports — feature modules
- src/layouts — AuthLayout, DashboardLayout
- src/pages — route-level components
- src/hooks — reusable hooks
- src/services — API client and feature services
- src/mocks — seed data and mock API
- src/utls — constants, formatters, validators
- src/styles — global tokens and theme

4. UI Blueprint: Screen by Screen

Screen	Essential UI	Practice focus
Login	Logo, email/password, remember me, error, submit	Form state, validation, navigation
Dashboard	Sidebar, topbar, KPI cards, activity, charts	Composition, mapping, derived data
Projects	Search, status filter, sort, grid/list toggle, pagination	State sync and reusable controls
Project Detail	Header, tabs, overview, board, members, activity	Nested routes and layouts
Kanban	Four columns, task cards, add task, drag/drop	Immutable nested state
Task Drawer	Editable fields, subtasks, comments, activity	Portal/modal, complex forms
Team	Member table, role dropdown, invite modal	RBAC and reusable table
Reports	Filters, charts, empty/loading states	Memoization and visualization
Settings	Profile, notification toggles, theme	Context and localStorage

Design system (build before pages)

- Colors: primary, success, warning, danger, neutral scale
- Spacing: 4, 8, 12, 16, 24, 32
- Typography: title, H1/H2/H3, body, caption
- Components: Button, Input, Select, Badge, Card, Avatar, Modal, Spinner, EmptyState
- States: default, hover, focus, disabled, loading, error, empty

UI rule Page ke andar raw button/input baar-baar mat likhiye. Pehle reusable UI primitive banaiye, phir feature components me compose kijiye.

5. Build Roadmap — 10 Phases

Phase	Time @2h/day	Outcome
Phase 0 — JavaScript readiness	2–3 days	Array methods, destructuring, spread, modules, async/await, immutability
Phase 1 — Setup + static shell	3–4 days	Vite, JSX, components, props, CSS, layout

Phase 2 — Dashboard data	5–6 days	useState, map, keys, events, conditional rendering
Phase 3 — Projects CRUD UI	7–8 days	Forms, lifting state, validation, filters
Phase 4 — Routing + auth	5–6 days	React Router, layouts, protected routes, Context
Phase 5 — Kanban + tasks	8–10 days	useReducer, complex state, custom hooks, drag/drop
Phase 6 — Team + RBAC	5–6 days	Permissions, reusable table/modal, composition
Phase 7 — Data layer	7–8 days	Mock async API, loading/error, TanStack Query
Phase 8 — Advanced performance	6–7 days	memo, useMemo/useCallback, lazy/Suspense, error boundary
Phase 9 — Testing + polish	7–9 days	Unit/integration tests, accessibility, responsive UI
Phase 10 — Real API	Later	Axios/fetch client, token, CRUD endpoints, cache invalidation

Phase 0 — JavaScript readiness

Duration: 2–3 days | Focus: Array methods, destructuring, spread, modules, async/await, immutability

- Revise map/filter/find/reduce/sort.
- Objects/arrays ko mutate kiye bina update karein.
- Promise, async/await, try/catch ka mini practice.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 1 — Setup + static shell

Duration: 3–4 days | Focus: Vite, JSX, components, props, CSS, layout

- Create Vite app and clean starter files.
- Build AppShell, Sidebar, Topbar and responsive layout.
- Create design tokens and 5 reusable UI primitives.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 2 — Dashboard data

Duration: 5–6 days | Focus: useState, map, keys, events, conditional rendering

- Mock dashboard data file banayein.
- Cards/activity ko map se render karein.
- Loading, empty and error UI manually simulate karein.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 3 — Projects CRUD UI

Duration: 7–8 days | Focus: Forms, lifting state, validation, filters

- Projects list/grid toggle and filters.
- Create/Edit Project modal with controlled form.
- Validation, delete confirmation and local persistence.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 4 — Routing + auth

Duration: 5–6 days | Focus: React Router, layouts, protected routes, Context

- Public/private routes and 404 page.
- AuthProvider + useAuth custom hook.
- Role-based navigation and redirect handling.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 5 — Kanban + tasks

Duration: 8–10 days | Focus: useReducer, complex state, custom hooks, drag/drop

- Project board with normalized task data.
- Task reducer actions: add/edit/delete/move.
- Task drawer, subtasks, comments and drag/drop.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).

- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 6 — Team + RBAC

Duration: 5–6 days | Focus: Permissions, reusable table/modal, composition

- Members table, search, role filter.
- Permission utility: can(user, action).
- Hide/disable actions with explanation.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 7 — Data layer

Duration: 7–8 days | Focus: Mock async API, loading/error, TanStack Query

- Replace direct mock imports with service functions.
- Promise delay and random failure simulation.
- TanStack Query: query, mutation, invalidate, optimistic update.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 8 — Advanced performance

Duration: 6–7 days | Focus: memo, useMemo/useCallback, lazy/Suspense, error boundary

- Profile before optimizing.
- Memoize expensive report calculation.
- Lazy-load reports/settings and add ErrorBoundary.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

Phase 9 — Testing + polish

Duration: 7–9 days | Focus: Unit/integration tests, accessibility, responsive UI

- Test behavior, not implementation.
- Keyboard navigation, labels, focus states, contrast.
- Mobile/tablet layouts and final refactor.

Definition of Done

- Feature refresh ke baad expected state maintain karti hai (jahan required).
- Loading, empty, error aur success states visible hain.
- Console me warning/error nahi; code readable aur reusable hai.

6. Detailed 12-Week Study Plan (2 hours/day)

Week	Learning	Deliverable
1	JS refresh + Vite + Git	Shell, sidebar, topbar
2	Components + props + styling	Design system + dashboard
3	State, lists, events	Dashboard interactions
4	Forms + validation	Projects CRUD UI
5	Router + layouts	All routes + 404
6	Context + auth	Protected routes + roles
7	useReducer + custom hooks	Task state engine
8	Kanban + task detail	Board complete
9	Async + service layer	Mock API states
10	TanStack Query + Redux comparison	Server cache + app state
11	Performance + testing	Lazy routes + key tests
12	Accessibility + responsive + review	Portfolio-ready frontend

Daily 2-hour formula

Time	Work
20 min	Concept revise + notes
70 min	Feature coding
20 min	Debug/refactor without AI
10 min	Commit + learning journal

Learning journal Daily 3 lines likhein: aaj kya banaya, kya bug mila, aur kal bina dekhe kya explain kar sakti hain.

7. State Management Decision Guide

State type	Example	Where to keep
Local UI	Modal open, input value	useState
Shared feature	Project filters	Lift state / Context
Complex transitions	Task board actions	useReducer
Global client state	Auth user, theme	Context or Redux
Server data	Projects from API	TanStack Query
URL state	Search/page/tab	Router search params

Rule of thumb

- State ko closest common parent me rakhein.
- Jo value calculate ho sakti hai use duplicate state me store na karein.
- Form state aur server data ko ek hi global store me dump na karein.
- useEffect ko data derivation ke liye nahi; external synchronization ke liye use karein.

8. Future API Contract (Backend-ready Frontend)

Method	Endpoint	Purpose
POST	/api/auth/login	Login + tokens
GET	/api/me	Current user
GET/POST	/api/projects	List/create projects
GET/PATCH/DELETE	/api/projects/:id	Project detail/update/delete
GET/POST	/api/projects/:id/tasks	Tasks list/create
PATCH	/api/tasks/:id	Update task/status
GET/POST	/api/tasks/:id/comments	Comments
GET	/api/team	Members

PATCH	/api/team/:id/role	Change role
-------	--------------------	-------------

Frontend service contract

- `projectService.getProjects(params)`
- `projectService.getProject(id)`
- `projectService.createProject(payload)`
- `taskService.updateTask(id, payload)`
- `authService.login(credentials)`
- `teamService.updateRole(memberId, role)`

Key architecture Components endpoint URLs ko directly call nahi karenge. Component → custom hook/query → service → API client flow rakhein. Isse mock service ko real backend se replace karna easy hoga.

9. Testing Plan

Test level	What to test	Example
Unit	Pure utilities/reducer	taskReducer MOVE_TASK
Component	UI behavior	Validation message appears
Integration	Multiple components	Create project → list updated
Route	Access rules	Viewer redirected
API mock	Loading/error/success	Projects request fails
E2E optional	Critical journey	Login → create → move task

Minimum test checklist

- Project form required fields
- Search/filter result
- Auth protected route
- Role permission action
- Task reducer add/edit/move/delete
- Loading, empty and error states
- Keyboard-accessible modal close/focus

10. Codex Prompt Playbook

Har prompt me context + current phase + constraints + exact output + verification zaroor dein. Codex ko ek baar me pura app generate karne ko na bolkar ek vertical slice dein.

Prompt 0 — Project planning

You are my React mentor and pair programmer. I am a Laravel developer learning React. We are building “WorkNest”, a multi-role project and task management dashboard. Do not write code yet. First create: (1) feature breakdown, (2) route map, (3) component tree, (4) data models, (5) state ownership map. Use JavaScript, Vite, React Router and CSS Modules. Backend is not available, so plan a replaceable mock service layer. Explain decisions in simple Hinglish.

Prompt 1 — Setup and shell

In the existing Vite React project, implement only the responsive application shell: Sidebar, Topbar, main content area and mobile navigation. Create reusable NavItem and Avatar components. Use CSS Modules and semantic HTML. Do not add Redux or API calls. Before editing, inspect the existing files and preserve unrelated code. After implementation, run lint/build and explain each created file.

Prompt 2 — Dashboard

Build the WorkNest dashboard using mock data. Create reusable StatCard, ProjectProgress, ActivityList and WorkloadSummary components. Practice props, map, keys, conditional rendering and derived values. Include loading, empty and populated states controlled by a demo switch. Do not use useEffect for calculated values. Add a short exercise list for me to modify the result myself.

Prompt 3 — Projects CRUD

Implement Projects page with search, status filter, sort, grid/list toggle, create/edit form modal and delete confirmation. Use controlled inputs and local state first. Keep ProjectForm reusable for create and edit. Persist demo data in localStorage through a custom hook. Add validation and accessible labels/focus behavior. Do not install a form library yet. Run tests/build.

Prompt 4 — Routing and auth

Add React Router with AuthLayout and DashboardLayout, public login route, protected application routes, nested project routes and 404 page. Create AuthContext and useAuth. Add four roles: admin, manager, member, viewer. Implement permission-based navigation. Use fake login only; keep the interface ready for a later authService. Explain route flow and redirects.

Prompt 5 — Kanban reducer

Design and implement a Kanban task board using useReducer. Define task state and explicit actions for ADD_TASK, UPDATE_TASK, DELETE_TASK and MOVE_TASK. Keep reducer pure and immutable. Build four columns and a task drawer. First show me the state shape and reducer plan; then implement. Add reducer unit

tests and explain why each immutable update works.

Prompt 6 — Async mock service

Refactor direct mock-data usage behind `projectService` and `taskService`. Each method should return a Promise, simulate latency and support controlled failure. Add loading, error, empty and retry UI. Components must not import raw mock data. Do not add TanStack Query yet. Explain how this service will later be replaced by Laravel or Spring Boot endpoints.

Prompt 7 — TanStack Query

Migrate project/task server-state handling to TanStack Query. Add query keys, queries, mutations, invalidation and one optimistic task-status update with rollback. Keep auth/theme outside server cache. Explain query cache lifecycle and show which earlier `useEffect/useState` code can now be removed.

Prompt 8 — Performance

Profile the current WorkNest app conceptually and identify actual re-render risks. Apply `React.memo`, `useMemo` or `useCallback` only where justified. Lazy-load Reports and Settings routes with `Suspense`. Add an `ErrorBoundary`. Show before/after reasoning; do not add memoization everywhere.

Prompt 9 — Testing

Using Vitest and React Testing Library, add behavior-focused tests for `ProjectForm` validation, project filtering, protected routes, role permissions and `taskReducer`. Mock network behavior with the simplest suitable approach. Avoid testing internal implementation details. Run the test suite and fix failures.

Prompt 10 — Code review

Act as a senior React reviewer. Inspect the WorkNest codebase without changing it first. Review component responsibility, state placement, `useEffect` misuse, accessibility, performance, naming, folder structure, error/loading/empty states and test gaps. Rank findings as critical/high/medium/low with file references. After I approve, fix one category at a time.

Prompt 11 — Real API integration

Integrate the existing backend APIs into WorkNest without changing UI behavior. First inspect the OpenAPI/Swagger contract. Create a centralized API client, environment-based base URL, auth token handling, typed/validated payload mapping, service methods and TanStack Query hooks. Handle 401, 403, 422 and 500 separately. Do not place `fetch/axios` calls directly in components. Migrate one feature at a time and keep mock mode available.

11. How to Use Codex Without Losing Practice

Wrong habit	Better habit
"Pura app bana do"	One feature + constraints + acceptance criteria
Generated code copy-paste	Predict first, then compare
Error आते ही AI	15 min debug: message, stack, data, hypothesis
No verification	Ask Codex to run lint/test/build
Every file changed at once	Small commits and reviewable diffs
Concept skipped	Ask "why this hook here?" and rewrite yourself

Recommended learning loop

1. PLAN: feature ko components/state/events me break karein.
2. ATTEMPT: basic version khud likhein.
3. ASK: specific blocker ya review Codex ko dein.
4. VERIFY: browser behavior + lint + tests.
5. EXPLAIN: code ko apne words me note karein.
6. EXTEND: ek requirement bina AI add karein.

12. Final Portfolio Acceptance Checklist

- ☐ Responsive UI desktop/tablet/mobile par usable hai.
- ☐ Public/private/nested routes work karte hain.
- ☐ At least four roles ke permissions visibly different hain.
- ☐ Projects CRUD, task CRUD aur Kanban movement complete hai.
- ☐ Forms accessible aur validation clear hai.
- ☐ Loading/error/empty/success states har async feature me hain.
- ☐ State ka ownership documented aur sensible hai.
- ☐ No unnecessary useEffect; no direct DOM manipulation.
- ☐ Routes lazy-loaded; major renders reviewed.
- ☐ Core reducer/forms/routes tests pass hain.
- ☐ README me setup, architecture, screenshots, decisions aur future API notes hain.
- ☐ Environment variables aur API base URL hard-coded nahi hain.

13. Interview Questions You Should Answer After Completion

1. Props vs state kya hai?

2. State lifting kab karte hain?
3. useEffect kab required hai aur kab nahi?
4. Controlled form kya hota hai?
5. Context aur Redux me difference?
6. Client state vs server state?
7. Reducer pure kyun hona chahiye?
8. React reconciliation aur keys ka role?
9. useMemo/useCallback kab harmful ho sakte hain?
10. Protected route security ka complete solution kyun nahi?
11. Optimistic update rollback kaise kaam karta hai?
12. Accessible modal ke requirements kya hain?

Your next action Phase 0 se start karein. Repository create karke Prompt 0 Codex ko dein; plan approve hone ke baad hi Prompt 1 chalayein. Har phase ka screenshot aur learning note README me add karein.